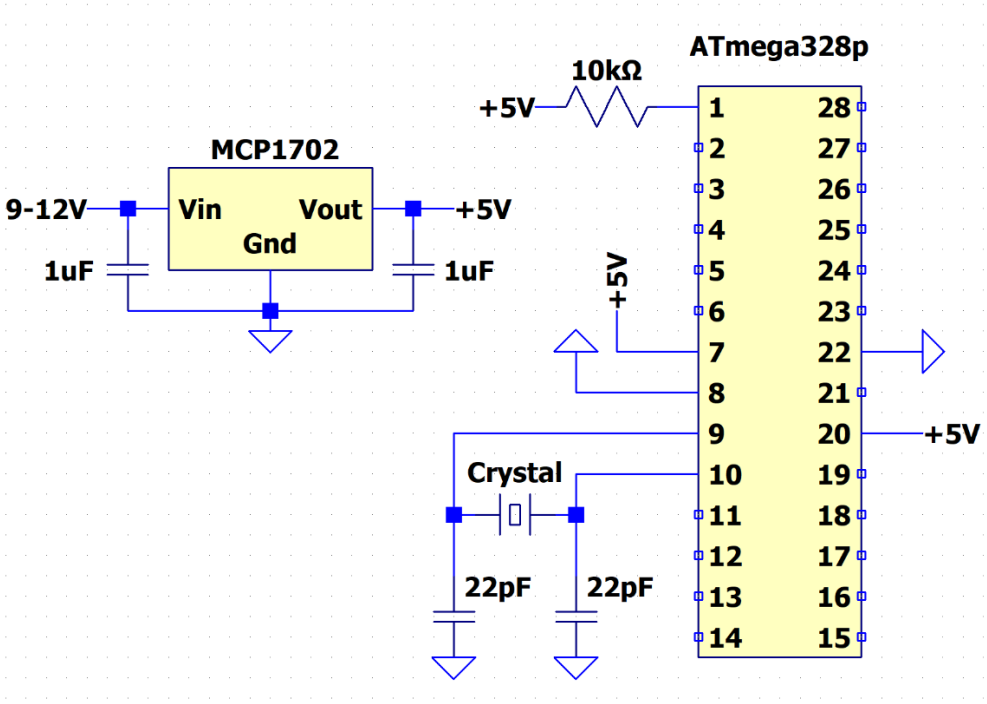
Programming Standalone ATmega328p

Steps for programming standalone ATmega328p

1. Step 1 - Building the circuit
    a. Connect the hardware needed for the ATmega328p to run without the Arduino Uno board. This includes the 16MHz crystal and its capacitors, the reset pin being pulled high with a 10kΩ resistor, and a stable 5V supply from your 5V LDO regulator (make sure to test your 5V supply before using it on the ATmega328p). Use the following figure as reference



2. Step 2a – Burning the bootloader
    a. Connect your Arduino board to your ATmega328p chip using the following connections:
        i. Arduino Uno <-> ATmega328p
        ii. Gnd <-> Gnd
        iii. D10 <-> Pin 1
        iv. D11 <-> Pin 17
        v. D12 <-> Pin 18
        vi. D13 <-> Pin 19
    b. In the Arduino IDE, make sure your software is up-to-date (click on Help -> Check for Arduino IDE Updates)
    c. Open the Arduino ISP sketch (File -> Examples -> 11.ArduinoISP ->Arduino ISP)

d. Plug in your Arduino Uno to your computer and select the correct COM port and device, and upload the ISP sketch to the Uno (like you typically upload code to the Uno)
e. Select the Board needed for your ATmega328p (Tools -> Board -> Arduino AVR Boards -> Arduino Duemilanove or Diecimila)
f. Select how you want to program this board (Tools -> Programmer -> Arduino as ISP)
g. Burn the Bootloader (Tools -> Burn Bootloader)

3. Step 2b – Troubleshooting the bootloader
    a. If you get an error message saying the device ID is wrong, something like:
        avrdude: Device signature = 0x1e9514 (probably m328)
        avrdude: Expected signature for ATmega328P is 1E 95 0F
    then your ATmega328p chip is actually an ATmega328 chip (notice the lack of the letter "p") and you need to change the expected device signature of the chip
    b. Locate the boards.txt file (for my install with the latest version of the Arduino IDE, it was located:
    "C:\Users\[username]\AppData\Local\Arduino15\packages\arduino\hardware\avr\1.8.6"
    c. Open the file with a basic text editor (like notepad), locate the line that has the target board for the ATmega328p (try searching "## Arduino Duemilanove or Diecimila w/ ATmega328P")
    d. Place the following text after the end of that entry and before the next entry:

    ## Arduino Duemilanove or Diecimila w/ ATmega328
    ## ---
    diecimila.menu.cpu.atmega328u=ATmega328

    diecimila.menu.cpu.atmega328u.upload.maximum_size=30720
    diecimila.menu.cpu.atmega328u.upload.maximum_data_size=2048
    diecimila.menu.cpu.atmega328u.upload.speed=57600

    diecimila.menu.cpu.atmega328u.bootloader.high_fuses=0xDA
    diecimila.menu.cpu.atmega328u.bootloader.extended_fuses=0xFD
    diecimila.menu.cpu.atmega328u.bootloader.file=atmega/ATmegaBOOT_168_atmega328.hex

    diecimila.menu.cpu.atmega328u.build.mcu=atmega328

    e. Your boards.txt file should look like this:

```
## Arduino Duemilanove or Diecimila w/ ATmega328P
## ----------------------------------------------
diecimila.menu.cpu.atmega328=ATmega328P

diecimila.menu.cpu.atmega328.upload.maximum_size=30720
diecimila.menu.cpu.atmega328.upload.maximum_data_size=2048
diecimila.menu.cpu.atmega328.upload.speed=57600

diecimila.menu.cpu.atmega328.bootloader.high_fuses=0xDA
diecimila.menu.cpu.atmega328.bootloader.extended_fuses=0xFD
diecimila.menu.cpu.atmega328.bootloader.file=atmega/ATmegaBOOT_168_atmega328.hex

diecimila.menu.cpu.atmega328.build.mcu=atmega328p

## Arduino Duemilanove or Diecimila w/ ATmega328
## ----------------------------------------------
diecimila.menu.cpu.atmega328u=ATmega328

diecimila.menu.cpu.atmega328u.upload.maximum_size=30720
diecimila.menu.cpu.atmega328u.upload.maximum_data_size=2048
diecimila.menu.cpu.atmega328u.upload.speed=57600

diecimila.menu.cpu.atmega328u.bootloader.high_fuses=0xDA
diecimila.menu.cpu.atmega328u.bootloader.extended_fuses=0xFD
diecimila.menu.cpu.atmega328u.bootloader.file=atmega/ATmegaBOOT_168_atmega328.hex

diecimila.menu.cpu.atmega328u.build.mcu=atmega328
```

f. Then, close out of the Arduino IDE, reopen the IDE, reload the board data (Tools -> Reload Board data), and restart step 2a of this document, however after part e, you'll want to select the new processor option of ATmega328 (Tools -> Processor -> ATmega328)

4. Step 3 - Uploading a program to the ATmega328p
    a. Connect your Arduino board to your ATmega328p chip using the following connections:
        i. Arduino Uno <-> ATmega328p
        ii. Gnd <-> Gnd
        iii. D10 <-> Pin 1
        iv. D11 <-> Pin 17
        v. D12 <-> Pin 18
        vi. D13 <-> Pin 19
    a. Open the sketch you want to upload to the ATmega328p
    b. Plug in your Arduino Uno and select the correct COM port and device for your Uno
    c. Select the Board needed for your ATmega328p (Tools -> Board -> Arduino AVR Boards -> Arduino Duemilanove or Diecimila)
    d. If you needed to add a new processor in Step 2b, select the processor you used before (the ATmega328), otherwise stay with the default of ATmega328p
    e. Select how you want to program this board (Tools -> Programmer -> Arduino as ISP)

f. Select Sketch > Upload Using Programmer

5. Step 4 – Using the Serial Monitor to Debug your code
    a. Once you move off of the Arduino Uno board, you'll quickly notice that it becomes much harder to debug your code without being able to use the Serial Monitor in the Arduino IDE. You could consider other forms of debugging (like always testing your code on the Uno first, or using LEDs with the ATmega328p to indicate your state), but you can still use the Serial Monitor on the IDE, you'll just need to route the Serial Print messages through the Uno.
    b. To send messages from the ATmega328p to the Uno and ultimately the Arduino IDE Serial Monitor, you'll need to make the following connections:
        i. Arduino Uno RX <-> ATmega328p RX
        ii. Arduino Uno TX <-> ATmega328p TX
        iii. Arduino Uno Reset <-> Gnd
        iv. Arduino Gnd <-> Atmega328p Gnd
    c. Making the RX and TX connections will route your messages to your Uno, and connecting the Uno Reset pin to ground will make sure that it isn't trying to run its own code but instead just acting as a relay for the messages.
    d. **Important Note:** These connections will have to be removed if you want to flash new code to the ATmega328p chip, however you can always put the connections back after flashing the code so continue to be able to use the Serial Monitor for debugging.
6. Resource - See the following diagram to map the various analog and digital pins of the Arduino to pins on the ATmega328p. Note that they are not the same pin numbers (i.e., Digital Pin 6 in your code is not pin 6 on the ATmega328p chip).

Pinout of ATmega328p